C2PA Asset Rubrics

C2PA Technical Working Group Conformance Task Force

Version 1.0, 2026-08-06

Table of Contents

1. Overview

The C2PA Conformance Program utilizes **Asset Rubrics** as a standardized framework to evaluate assets after the completion of the formal validation process. While C2PA validation confirms the structural integrity and cryptographic validity of a manifest store (e.g., hash matching and signature verification), rubrics evaluate the resulting data against normative requirements and classification criteria.

Asset rubrics enable automated, consistency-driven evaluation of the `crJSON` output produced by Validator Products.

2. Objectives of Asset Rubrics

Asset rubrics serve two primary functions within the C2PA ecosystem:

1. **Conformance Verification**: Rubrics enable the Conformance Program to verify that an asset produced by a Generator Product adheres to the normative requirements of the C2PA Specification and the Conformance Program itself.
2. **Asset Classification and Labeling**: Rubrics enable Validator Products to analyze asset history and identify provenance signals. This classification facilitates the generation of user-facing labels (e.g., differentiating camera captures from generative content).

By encoding evaluation logic into YAML definitions utilizing json-formula expressions, the program ensures objective and uniform evaluation across disparate implementations.

3. Asset Rubric Specification and Serialisation

3.1. Introduction

3.1.1. General

An asset rubric shall be expressed as a series of YAML documents, where the first document shall be the information block that contains metadata about the asset rubric. Any subsequent documents contain a series of statements, data blocks or both. These statements and data blocks define the expressions to be evaluated against the crJSON produced from a Content Credential. Each statement contains information that will be output into the produced report, such as a description of the statement, the expression to be evaluated, and the text to be included in the report. Each data block enables using templates and expressions to provide additional context or information to be output into the report.

NOTE	A YAML document is a section of a YAML file that is separated from the next section by a YAML document separator, which is a line containing three dashes (`---`). The first document in a YAML file is the one between the first and second separators.

3.1.2. Expressions

Expressions are strings of json-formula, an expression grammar that operates on JSON documents. In the context of asset rubrics, expressions are used to evaluate crJSON. As such, they evaluate the provided crJSON as input and produce a single value as an output. The output value can be binary, numeric (floating point), textual (string), a structured object (such as an array or object), or a URI

reference which can be internal (JUMBF reference) or external (URL).

For example, the simple expression `manifests[0].status.content == "assertion.dataHash.match"` could be used to determine if the content status of the active manifest matches the expected value of `assertion.dataHash.match`. The output of this expression would be a boolean value indicating whether the condition is true or false.

3.1.3. Multilingual text

When a field is expressed in multiple languages, these multilingual fields use a mapping with [BCP47] language tags as key and the text in the corresponding language as value as in Example of multilingual fields. These types are indicated as `ML Dict`.

Example of multilingual fields

```
field_name:
  en: Text in English.
  es: Text in Spanish.
  fr: Text in French.
```

3.2. Templates

Many aspects of an asset rubric can contain template values that are replaced with the actual values when the report is generated. The template values are enclosed in double curly braces (`{{` and `}}`) and can refer to the output value of the statement, the metadata of the asset rubric, or other variables defined in the asset rubric. For example, `{{id}}` refers to the result of an expression statement with that `id`, and `{{rubric_metadata.name}}` refers to the `name` field of the `rubric_metadata` mapping from the information block.

Examples of using templates illustrates the use of templates in an asset rubric.

Examples of using templates

```
reportText:
  "true":
    en: This media asset is compliant with '({{rubric_metadata.name}},
{{rubric_metadata.version}})'.

- block:
    name: "test_map"
    value:
      alg: "{{ asset_info.alg }}"
      hash: "{{ asset_info.hash }}"
```

A template value can also be a full blown expression, which allows for not just simple value replacement but also more complex evaluations. These template expressions look similar to standard templates, in that they are enclosed in double curly braces (`{{` and `}}`), but immediately following the opening curly brace is the keyword `expr`, followed by the expression to be evaluated (enclosed in double quotes) and then the closing brace. For example, the template `{{expr "_dateOfWeek()"}}` would call the expression `_dateOfWeek()` defined in the information block and replace the template with the result of that expression.

The other common use for template expressions is to be able to refer to fields and values in the crJSON, such as `{{expr "manifests[0].signature.issuer.CN"}}`, which would return the CN (Common Name) from the signature on the active Trust Manifest.

NOTE By combining global expressions with template expressions, it is possible to create complex reports that adapt to the context of the media asset and the asset rubric.

3.3. Information block

3.3.1. Metadata

The asset rubric information block, which shall be the first document in the YAML file, shall consist of a YAML mapping that provides additional information about the specific asset rubric. It shall contain at least one mapping with the key `rubric_metadata`, which shall only contain the keys listed in asset rubric metadata, such as its name and date of issue. Some of the keys are mandatory and therefore shall always be present. Its function is to provide a structured way to include metadata about the asset rubric.

The information block may also contain additional mappings that provide further context or configuration for the asset rubric, as described in Custom information. Additional mappings may either be defined by this document or may be custom to a specific use case.

Table 1. asset rubric metadata

Key	Value	Type	Mandatory
name	Name of the asset rubric.	`String`	yes
issuer	Name of the issuer of the asset rubric.	`String`	yes
date	Date when the asset rubric was issued.	date (as defined in [ISO8601])	yes
version	Version number of the asset rubric.	`String` (formatted as per [SEMVER])	yes
language	Default of the text in the asset rubric for non multilingual fields.	[BCP47] language tag	no

A simple example of the metadata mapping is shown in Example of metadata in an asset rubric.

Example of metadata in an asset rubric

```
rubric_metadata:
  name: Testing Profile
  issuer: Someone
  date: 2025-06-17T22:44:49.717Z
  version: 2.0.0
  language: en
```

3.3.2. Custom information

Additional mappings may be added to the information block to provide further context or configuration

for the asset rubric or to provide values for use in expressions. These mappings are not defined by this document and may vary depending on the use case. The keys for these mappings shall be unique within the asset rubric, and shall use appropriate namespacing syntax. Some examples of custom information are shown in Example of custom information in an asset rubric.

Example of custom information in an asset rubric

```
"foo:info":
  description: This is a sample camera profile for testing purposes
  magic_number: 1234567890

"my:example":
  description: "This is a test example"
  myDate: "{{ rubric_metadata.date }}"
  myNumber: |
    {{ expr "5*5"}}
```

3.3.3. Use in templates and expressions

All of the mappings present in the information block are made available to the templates and expressions in the asset rubric. For example, it is possible to use the metadata in templates to provide context for the evaluation of the crJSON.

It is strongly recommended that an asset rubric output the entire `rubric_metadata` mapping into a data block in the report, also named `rubric_metadata` using a data block as shown in Example of outputting the metadata in a data block.

Example of outputting the metadata in a data block

```
# Output the metadata
- block:
    name: "rubric_metadata"
    value: |
      {{ expr "@.rubric_metadata" }}
```

To output a custom mapping, such as `my:Example` from the information block via a data block, the following expression statement in Example of outputting a custom mapping in a data block could be used.

Example of outputting a custom mapping in a data block

```
- block:
    name: "myExample"
    value: |
      {{expr "@.'my:Example'"}}
```

3.3.4. Globals

General

In order to provide a common context for the evaluation of the expressions in the asset rubric, the information block may contain a `variables` mapping as well as an `expressions` mapping.

Variables

The `variables` mapping contains key-value pairs that define variables that can be used in the expressions in the asset rubric. The keys for these variables shall all begin with a `$` and shall be unique within the asset rubric. The values of the variables can be any valid JSON value, including strings, numbers, booleans, arrays, and objects. Some examples of variables are shown in Example of variables in the asset rubric.

Example of variables in the asset rubric

```
variables:
  "$creation_date": "2023-10-01T12:00:00Z"
  "$creator": "John Doe"
  "$days": [ "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday",
"Sunday" ]
```

Expressions

The `expressions` mapping contains key-value pairs that define expressions that can be used in the asset rubric. The keys for these expressions shall all begin with a `_` and be unique within the asset rubric. The values of the expressions are strings that follow the json-formula grammar. Some examples of expressions are shown in Example of expressions in the asset rubric.

Example of expressions in the asset rubric

```
expressions:
  _isUnmodified: '@.manifests[0].status.content == "assertion.dataHash.match"'
  _isAIGC:
contains(@.manifests[0].assertions.'c2pa.actions'.actions[0].digitalSourceType,
"trainedAlgorithmicMedia")
  _dateOfWeek: "value($days, weekday(now(), 3))"
```

Parameterized expressions

Named expressions may accept arguments (modelled after `registerWithParams`). Argument 0 (the first positional argument at the call site) becomes `@` inside the expression body; arguments 1 through N are collected into a single array global `$args`, so `$args[0]` is argument 1, `$args[1]` is argument 2, and so on. Zero-argument expressions are called with empty parentheses and evaluate with the caller's data context as `@`.

```
expressions:
  # 0-arg: @ inside the body is whatever the calling expression's @ is
  _actionsV2: "manifests[0].assertions.'c2pa.actions.v2'.actions || `[]`"

  # 1-arg: @ is the manifest passed at the call site
  _manifest_signalActions: "@.assertions.'c2pa.actions.v2'.actions"
```

```
  # 2-arg: @ is the manifest, $args[0] is the digitalSourceType URI string
  _manifest_createdWithDst: |-
    length(_manifest_signalActions(@)[? action == "c2pa.created" && digitalSourceType
== $args[0]]) > 0
```

```
# 1-arg call — pass the active manifest explicitly
expression: "_manifest_signalActions(manifests[0])"

# 2-arg call — manifest is @, DST URI literal becomes $args[0]
expression: '_manifest_createdWithDst(manifests[0],
"http://c2pa.org/digitalsourcetype/empty")'
```

NOTE

Unlike `@`, the `$args` global array is **not** shadowed inside filter projections (`[?condition]`). Within a filter, `@` is re-bound to the current array element, making any outer `@` inaccessible. `$args[0]`, `$args[1]`, etc., however, remain available at every nesting depth because `$args` is a global. When a named expression needs to reference a scalar value inside a filter while also iterating over a collection, pass the scalar as argument 1 (→ `$args[0]`) and the collection as argument 0 (→ `@`).

3.3.5. Includes

An asset rubric may include other asset rubric snippets using the `include` directive. This allows for modular asset rubrics that can be reused across different use cases. The `include` directive is specified as shown in Example of including an asset rubric snippet.

Example of including an asset rubric snippet

```
include:
  - path/to/other/asset_profile.yml
```

Each entry in the array is a path (either absolute or relative) to a YAML file containing the asset rubric snippet. The first document in the YAML, the information block, will be appended to the first document in the including profile - this enables having standard and custom metadata values, variables and expressions included. If there is any naming conflict, the values from the current (including) asset rubric will take precedence over anything being included. If two included files use the same identifier for a variable or expression, the value from the last included profile will be used.

Each subsequent document in the YAML will be appended to the including asset rubric, in the order they appear in the list of includes.

3.4. Data Blocks

An asset rubric may contain one or more data blocks, in any YAML document except the first, which are used to extract additional information from the crJSON or other sources.

NOTE

Data Blocks are currently optional and not implemented in the reference Python evaluator.

Each data block is represented as a mapping with the key `block` and contains a pair of required key-

value pairs:

`name`
: a field which will be the name of the mapping in the report,

`value`
: a field that can be of any valid YAML types (e.g., scalars, arrays or mappings) and can contain template values that are replaced with the actual values when the report is generated.

For example, the data block in Example of a data block in a profile extracts the algorithm and hash from the `asset_info` Trust Indicator in the crJSON into a data block named `asset_info` as seen in the report in Example output of that data block in the report.

Example of a data block in a profile

```
- block:
    name: "asset_info"
    value:
      alg: "{{ asset_info.alg }}"
      hash: "{{ asset_info.hash }}"
```

Example output of that data block in the report

```
asset_info:
  alg: sha256
  hash: na6lb3F/uIdiAhZtZp4Oa2aNCj1UvcHVxx/p5ISE2AA=
  myNumber: 100
```

An example data block that extracts the set of assertion statuses from the active Trust Record (manifest) in the Trust Indicator Set into a data block named `assertion_status` in the report could look like [example_data_block_in_profile_with_assertion_status], with the output in the report as shown in Example output of that data block in the report.

Example of a data block in a profile

```
- block:
    name: "assertion_status"
    value: |
      {{ expr "manifests[0].'claim.v2'.assertion_status" }}
```

Example output of that data block in the report

```
assertion_status:
  c2pa.hash.data: assertion.hashedURI.match
  c2pa.actions.v2: assertion.hashedURI.match
```

3.5. Statements

3.5.1. General

Following the information block document is one or more documents known as a statement section (or just section for short). Each section contains a list of statements, where each statement is represented as mappings with predefined fields.

There are two types of statements that can be present, in any order or combination, in a section: information statements and expression statements. Information statements are used to provide additional context or information about the section, while expression statements are used to evaluate expressions against the crJSON. Both types of statements provide a way to include human-readable report text that will be included in the associated report.

3.5.2. Statement identifiers

General

One of the fields that shall be present in all statements is the statement identifier (`id`). It shall have a value which can be any valid JSON identifier. If the identifier begins with the prefix `c2pa:`, then it represents one of the predefined identifiers described in Predefined statement IDs.

The identifier shall be unique within the asset rubric, as it can be used in an expression to refer to the result of the evaluation of the statement. Example of using a statement identifier in an expression shows an example of how to use the identifier in an expression.

Example of using a statement identifier in an expression

```
id: c2pa:signal_compliance
description: is the asset compliant with this profile?
expression: |
  @.profile.aigc && @.profile.declaration_only
reportText:
  "true":
    en: |
      Compliance Status: {{profile.c2pa:signal_compliance}}
  "false":
    en: This media asset is not compliant with this profile.
```

NOTE Example of using a statement identifier in an expression is interesting because it is self-referential, where the template being evaluated as part of the `reportText` references the result of the statement's expression.

Predefined statement IDs

This document defines some reserved statement identifiers that are used to signal that an expression statement is serving a particular purpose. Predefined statement IDs gives an overview of these predefined IDs and a description of their purpose.

Table 2. Predefined statement IDs

ID	Purpose	Type	Mandatory
c2pa:signal_compliance	Expression that produces a binary output that signals the overall compliance/signal outcome of a media asset.	`Boolean`	no

3.5.3. Use of Templates

The text in the report that is generated by a statement, as specified in the `reportText` field of a statement, can contain template values that are replaced with the actual values when the report is generated.

3.5.4. Information statements

An information statement is used to provide some explicit text at a specific point in the report, without the need to evaluate any expression. The most common use for one is to provide a title and description of each section, which is why there should be an information statement as the first statement in any section. Structure of information statements gives an overview of the structure of an information statement.

NOTE Information Statements are currently optional and not implemented in the reference Python evaluator (they are skipped during execution).

Table 3. Structure of information statements

Key	Value	Type	Mandatory
id	Identifier that uniquely identifies the statement within the asset rubric.	`String ([a-zA-Z0-9_.-]*)`	yes
description	A human readable description of the statement or section that will not be taken over in associated reports.	`String`	no
title	Section title that will be copied in the reports, if present.	`String` | `ML Dict`	no
reportText	Text that is copied in the reports, providing additional context or information.	`String` | `ML Dict`	yes

3.5.5. Expression statements

An expression statement is used to evaluate an expression against the crJSON and produce a value that can be included in the associated report. Each expression statement shall have an identifier (`id`), an expression to be evaluated, and the text to be included in the report. The output value of the expression shall also be included in the report, and also recorded back into the crJSON with the same identifier as the statement, as part of the `profile` section of the crJSON. For example, if the identifier of the expression statement is `isAIGC`, then the output value of the expression can be referenced in a future expression via `@.profile.isAIGC`.

NOTE If an expression cannot be evaluated, the output value of the expression is `null` (as per json-formula).

Structure of expression statements gives an overview of the structure of expression statements.

Table 4. Structure of expression statements

Key	Value	Type	Mandatory
id	Identifier that uniquely identifies the statement within the asset rubric.	`String ([a-zA-Z0-9_.-]*)`	yes
description	A human readable description of the statement that will not be taken over in associated reports.	`String` | `ML Dict`	no

Key	Value	Type	Mandatory
expression	Expression/formula to be evaluated against the Trust Indicator Set.	String (json-formula)	yes
reportText	Text that goes in the associated report to explain the output value of this statement to an end user. Can be a simple string, a multi-language dictionary, or a mapping keyed by 'true'/'false' to strings/ML dicts.	`String` | `ML Dict` | `Outcome Mapping`	yes
failIfMatched	When true, negates the evaluation outcome (useful for blocklist/violation rules where matching an expression indicates failure).	`Boolean`	no

4. Categories of Rubrics

The evaluation framework is divided into three functional categories:

4.1. 1. Integrity Rubrics

Integrity rubrics assess whether an asset is structurally well-formed and trusted. These rubrics evaluate the `validationResults` to identify errors or anomalies in the validation output.

```
- id: validation:well_formed_success
  description: Check if the asset has any structural or malformation failures (from
the well-formed list)
  failIfMatched: true
  expression: '_validationResults().failure[?contains($well_formed_error_codes,
code)].code'
```

4.2. 2. Conformance Rubrics

Conformance rubrics verify adherence to specific versioned requirements of the C2PA Specification or the Conformance Program.

For example, verifying the mandatory presence of an inception action (`c2pa.created` or `c2pa.opened`) as the first action in the first actions assertion in created_assertions:

```
- id: validation:inception_action_position
  expression: |
    endsWith(notNull(_createdAssertions()[?contains(url, "c2pa.actions")] | [0].url,
""), "c2pa.actions.v2") &&
    (length(_actionsV2()[?action == "c2pa.created"]) == 0 ||
      contains(`["c2pa.created", "c2pa.opened"]`, _actionsV2()[0].action))
```

4.2.1. Version-Gated Program Policies

Certain policy rules established by Conformance Program 0.2 (such as `validation:mandatory_spec_version`

and `validation:no_dst_for_opened_action` in `conformance-program-0.2.yml`) utilize SemVer conditional gates evaluating `$expected_spec_version`. This design ensures that Program 0.2 rules targeting Spec 2.4+ features are strictly enforced for Spec 2.4+ submissions, while being automatically bypassed when evaluating assets submitted against earlier specification versions (e.g., Spec 2.2).

4.3. 3. Signals Rubrics

Signals rubrics analyze the manifest history (actions and digital source types) to evaluate the provenance of the asset. Unlike conformance rubrics that evaluate the asset as a whole, the signals rubric evaluates each manifest in the provenance chain **locally** to determine the signals associated with that specific node in the Directed Acyclic Graph (DAG) that represents the manifest store and the relationships between the manifests in it.

5. Signals Taxonomy: Inception vs. Transformation

The Signals Rubric distinguishes between **Inception Signals** (the origin of the asset) and **Transformation Signals** (modifications applied post-inception).

5.1. Inception Signals

Inception signals identify the initial creation state of the asset. These are triggered by a `c2pa.created` action accompanied by a specific `digitalSourceType`, or by the absence of an active manifest in an ingredient. Examples include:

- **Captured Media**: Initial capture from a physical device (camera).
- **Fully GenAI Media**: Content generated entirely by generative models.
- **Blank Canvas**: Initialization of an empty workspace, for example by invoking File > New in a creative app.

5.2. Transformation Signals

Transformation signals analyze modifications applied during the asset lifecycle. These are sub-categorized into **Non-Editorial** and **Perceptible** transformations.

5.2.1. Non-Editorial Transformations

Mechanical operations that do not represent a perceptible modification. These are defined by explicit allow-lists and include operations like format conversion, proportional resizing, and metadata editing.

5.2.2. Perceptible Transformations

Operations that alter visual or auditory content. Any action not specified on the non-editorial allow-list is classified as a perceptible transformation.

> "Changes content perceptively" means the content has been changed in some way, and that change is visible/audible to an ordinary viewer in ordinary circumstances. For example, color correction is generally visible; insertion of

> an invisible watermark is not generally visible.

When a perceptible transformation is detected, the rubric evaluates the associated `digitalSourceType` to classify the involvement of generative AI in the edit:

- **Perceptible GenAI**: Transformations driven by generative AI.
- **Perceptible Non-GenAI**: Manual interventions where the `digitalSourceType` indicates use of non-generative tools.
- **Perceptible Unknown / Possibly GenAI**: Ambiguous transformations utilizing generic terms that may contain generative synthesis.

6. Detailed Signals Criteria

The following section explains what each of the signals in `asset-rubric-signals-local.yml` is based on its json-formula criteria, described in natural language.

6.1. Inception Signals

- `inception:signal_blankCanvas`: Triggers if the manifest contains a `c2pa.created` action where the `digitalSourceType` is set to `http://c2pa.org/digitalsourcetype/empty`. This indicates the asset was created from an empty workspace.
- `inception:signal_capturedMedia`: Triggers if the manifest contains a `c2pa.created` action where the `digitalSourceType` is set to either `digitalCapture` or `computationalCapture`. This indicates the asset was captured from a physical device (e.g., a digital camera, a smartphone camera, a microphone or an audio interface connected to a Digital Audio Workstation).
- `inception:signal_capturedMediaStitched`: Triggers if the manifest contains a `c2pa.created` action where the `digitalSourceType` is set to `compositeCapture`. This indicates the asset was stitched from multiple captures (e.g., a panorama).
- `inception:signal_compositionMayContainGenAI`: Triggers if the manifest contains a `c2pa.created` action where the `digitalSourceType` is set to `composite`. This indicates a composition that may contain GenAI content.
- `inception:signal_fullyGenAIMedia`: Triggers if the manifest contains a `c2pa.created` action where the `digitalSourceType` is set to `trainedAlgorithmicMedia`. This indicates the asset was generated entirely by generative models.
- `inception:signal_mediaUnknownProvenance`: Triggers if the manifest contains an ingredient that does not have an `activeManifest`. This indicates that the asset was created by combining media of unknown provenance.
- `inception:signal_nonGenAIDigitalCreation`: Triggers if the manifest contains a `c2pa.created` action where the `digitalSourceType` is set to `digitalCreation`. This indicates a digital creation without GenAI.
- `inception:signal_partlyGenAICreation`: Triggers if the manifest contains a `c2pa.created` action where the `digitalSourceType` is set to `compositeWithTrainedAlgorithmicMedia`. This indicates a creation that partly contains GenAI content.
- `inception:signal_screenCaptureMayContainGenAI`: Triggers if the manifest contains a `c2pa.created` action where the `digitalSourceType` is set to `screenCapture`. This indicates a screen capture which may contain GenAI content.

6.2. Transformation Signals

- `transformation:signal_perceptibleGenAI`: Triggers if the manifest contains any action (other than creation or mechanical operations) where the `digitalSourceType` is one of the GenAI values: `trainedAlgorithmicMedia`, `compositeWithTrainedAlgorithmicMedia`, `compositeSynthetic`, or `trainedAlgorithmicData`. This indicates perceptible edits performed using GenAI (e.g., AI generated remixing of an existing recording).
- `transformation:signal_perceptibleNonGenAI`: Triggers if the manifest contains any action where `digitalSourceType` is present but is NOT one of the GenAI values listed above. This indicates manual perceptible edits.
- `transformation:signal_perceptiblePossiblyGenAI`: Triggers if the manifest contains any action where `digitalSourceType` is `digitalArt` or `composite`. These are ambiguous terms that might imply GenAI usage.
- `transformation:signal_nonEditorial`: Triggers if the manifest contains actions that are in the allow-list of non-editorial actions (e.g., `c2pa.converted`, `c2pa.published`, `c2pa.resized.proportional`, etc.). These are mechanical operations that do not alter the asset's content.
- `transformation:signal_ambiguousActions`: Triggers if the manifest has an ambiguous actions state (e.g. custom action without namespace, empty actions list, or `allActionsIncluded` is not true).

7. Utilizing Signals Rubrics for Generator and Validator Products

The local signals rubric (`asset-rubric-signals-local.yml`) serves as the definitive reference for how the C2PA Conformance Program classifies asset provenance. It is utilized by both Generator and Validator Products:

- **Generator Products**: Applicants building Generator Products utilize the rubric as a **blueprint** to understand how specific combinations of actions and `digitalSourceType` values are interpreted downstream. By aligning their manifest generation logic with the rubric, Generator Products ensure their assets emit the desired provenance signals.
- **Validator Products**: Applicants building Validator Products should utilize the rubric **unmodified** to ensure objective, conformant, and consistent classification of asset provenance across the C2PA ecosystem.

The program provides evaluation scripts accessible to both parties to verify that generated assets trigger the intended signals.

7.1. Evaluator Usage

The script `c2pa_signals_rubric_evaluator.py` (located in the `asset-rubrics` directory) evaluates an asset's data against a rubric and generates a classification report.

```
python3 c2pa_signals_rubric_evaluator.py asset-rubric-signals-local.yml
asset_output.json
```

The evaluator processes the `asset-rubric-signals-local.yml` and outputs a JSON containing evaluation results grouped by manifest node in the DAG. For each manifest, it outputs:

- `assertedBy`: The subject organization and common name from the signing certificate.
- `dc:format`: The format/MIME type of the asset at that state (if available).
- `localInceptions`: A list of inception signals detected in that manifest.
- `localTransformations`: A list of transformation signals detected in that manifest.
- `ingredients`: A list of parent manifests that contributed to this manifest.

Validator Products can analyze this structured DAG output to render precise provenance data to the end user.

8. Rubric Composition and Build System

To improve maintainability, the source files for the rubrics are modularized and stored in the `composables/` directory. Monolithic rubrics are generated from these components using Python build scripts. The version for all compiled rubrics is managed centrally via `VERSION`.

8.1. Project Structure

- `VERSION`: Stores the shared rubric version string (e.g., `1.0`).
- `composables/`: Contains partial YAML definitions (e.g., `signal-*.yml`, `integrity.yml`, `conformance-*.yml`).
- `build_local_signals_rubric.py`: Combines signals components into `asset-rubric-signals-local.yml`.
- `build_conformance_rubrics.py`: Combines program and spec components into the compiled conformance rubrics:
  - `asset-rubric-conformance0.1-spec2.2.yml` (Conformance Program 0.1 / Spec 2.2)
  - `asset-rubric-conformance0.2-spec2.2.yml` (Conformance Program 0.2 / Spec 2.2)
  - `asset-rubric-conformance0.2-spec2.4.yml` (Conformance Program 0.2 / Spec 2.4)
- `build_integrity_rubric.py`: Generates the standalone `asset-rubric-integrity.yml` from `composables/integrity.yml`.
- `build_rubrics.sh`: Shell script that runs all builders to regenerate all monolithic rubrics.

8.1.1. Dynamic Variable Injection

When `build_conformance_rubrics.py` compiles a specification target, it dynamically maps version-specific variables (e.g., `$allowed_assertions_v24`, `$allowed_actions_v24`, `$deprecated_assertion_labels_v24`) to generic aliases (`$allowed_assertions`, `$allowed_actions`, `$deprecated_assertion_labels`). This enables rules inherited from Spec 2.2 to automatically evaluate against expanded Spec 2.4 registries without code duplication.

8.2. Generating Rubrics

When modifying rules or updating `VERSION`, developers should run the orchestrator script to regenerate all monolithic rubrics:

```
./build_rubrics.sh
```

Or run individual build scripts:

```
python3 build_local_signals_rubric.py
python3 build_conformance_rubrics.py
python3 build_integrity_rubric.py
```

IMPORTANT	For technical specifications regarding rubric syntax, schema definitions, and expression evaluation, refer to Section 2 ("Asset Rubric Specification and Serialisation") of this document. The examples provided herein are simplified for conceptual comprehension.

Appendix A: Normative References

- **BCP47**: IETF Tags for Identifying Languages, https://tools.ietf.org/html/bcp47
- **ISO8601**: Data elements and interchange formats – Information interchange – Representation of dates and times, https://www.iso.org/iso-8601-date-and-time-format.html
- **SEMVER**: Semantic Versioning 2.0.0, https://semver.org/

Appendix B: C2PA Rules Matrix by Specification and Program Version

This appendix provides an independent breakdown of all validation rules evaluated across **C2PA Specification Versions** and **Conformance Program Versions**, showing explicit inheritance and replacement mechanics within each layer.

B.1. C2PA Specification Version Layer

B.1.1. Spec Version 2.2 (`conformance-spec-2.2.yml`)

Technical requirements established by C2PA Specification v2.2 (15 Rules).

#	Rule ID	Spec Section	Description	Source / Status
1	`validation:inception_action_position`	Spec 2.2 §18.14.2	Require inception action (`c2pa.created` / `c2pa.opened`) to be the first action in the first created actions assertion	Defined in Spec 2.2
2	`validation:mandatory_dst_for_created_action`	Spec 2.2 §18.14.2, Spec 2.2 §18.14.4.5	Require `digitalSourceType` on all `c2pa.created` actions	Defined in Spec 2.2
3	`validation:active_manifest_urn`	Spec 2.2 §8.1, Spec 2.2 §10.2.1	Require active manifest URN to use the standard `urn:c2pa:` prefix (prohibit legacy `urn:uuid`)	Defined in Spec 2.2
4	`validation:no_deprecated_assertions`	Spec 2.2 §6.3, App C.1	Flag deprecated standard assertion labels (e.g. `stds.`)	Defined in Spec 2.2
5	`validation:no_unsupported_assertions`	Spec 2.2 §6.2, Spec 2.2 §6.2.2, Spec 2.2 §6.3	Enforce that all standard assertions belong to the Spec 2.2 allow-list	Defined in Spec 2.2

#	Rule ID	Spec Section	Description	Source / Status
6	`validation:no_deprecated_actions`	Spec 2.2 §18.14.11, App C.1	Flag deprecated action types	Defined in Spec 2.2
7	`validation:no_unsupported_actions`	Spec 2.2 §18.14, Spec 2.2 §18.14.12	Enforce that all standard actions belong to the Spec 2.2 allow-list	Defined in Spec 2.2
8	`validation:no_unrecognized_custom_assertions`	Spec 2.2 §6.2.2	Require reverse-domain namespacing (must contain a dot `.`) for custom assertions	Defined in Spec 2.2
9	`validation:no_unrecognized_custom_actions`	Spec 2.2 §18.14, Spec 2.2 §18.14.12	Require reverse-domain namespacing (must contain a dot `.`) for custom actions	Defined in Spec 2.2
10	`validation:no_unrecognized_custom_action_parameters`	Spec 2.2 §18.14.4.7	Require reverse-domain namespacing (must contain a dot `.`) for custom action parameters	Defined in Spec 2.2
11	`validation:discouraged_legacy_action_parameters`	Spec 2.2 §18.14.4.1, Spec 2.2 §18.14.12	Warn if legacy `description` or singular `ingredient` fields are placed inside `parameters`	Defined in Spec 2.2 (Warn)
12	`validation:review_ratings_datasource`	Spec 2.2 §18.3.3	Prevent invalid co-occurrence of `reviewRatings` with human entry `dataSource`	Defined in Spec 2.2
13	`validation:ingredient_relationship_values`	Spec 2.2 §18.15.3	Require valid ingredient relationship values (`parentOf`, `componentOf`, `inputTo`)	Defined in Spec 2.2
14	`validation:ingredient_v3_no_active_manifest`	Spec 2.2 §18.15.12.4.3	Ensure flat ingredients (no `activeManifest`) omit `validationResults`	Defined in Spec 2.2
15	`validation:ingredient_v3_mandatory_validation_results`	Spec 2.2 §18.15.12.4.3	Ensure linked ingredients (with `activeManifest`) include `validationResults`	Defined in Spec 2.2

B.1.2. Spec Version 2.4 (`conformance-spec-2.2.yml` + `conformance-spec-2.4.yml`)

Technical requirements for C2PA Specification v2.4 (23 Rules Total: 15 inherited from Spec 2.2 + 8 new/replacing rules in Spec 2.4).

#	Rule ID	Spec Section	Description	Source / Upgrades & Replacements
1	`validation:inception_action_position`	Spec 2.4 §18.15.3	Inception action must be first action in first created actions assertion	Inherited from Spec 2.2
2	`validation:mandatory_dst_for_created_action`	Spec 2.4 §18.15.4	Require `digitalSourceType` on all `c2pa.created` actions	Inherited from Spec 2.2
3	`validation:active_manifest_urn`	Spec 2.4 §8.4.2, Spec 2.4 §10.2.1	Active manifest URN must use `urn:c2pa:` prefix	Inherited from Spec 2.2

#	Rule ID	Spec Section	Description	Source / Upgrades & Replacements
4	`validation:no_deprecated_assertions`	Spec 2.4 §6.3	Flag deprecated standard assertion labels	Inherited from Spec 2.2 (Upgraded to v2.4 deprecations)
5	`validation:no_unsupported_assertions`	Spec 2.4 §6.2.2	Enforce assertion allow-list	Inherited from Spec 2.2 (Upgraded to v2.4 allow-list)
6	`validation:no_deprecated_actions`	Spec 2.4 §18.15.11	Flag deprecated action types	Inherited from Spec 2.2 (Upgraded to v2.4 deprecations)
7	`validation:no_unsupported_actions`	Spec 2.4 §18.15.4	Enforce action allow-list	Inherited from Spec 2.2 (Upgraded to v2.4 allow-list)
8	`validation:no_unrecognized_custom_assertions`	Spec 2.4 §6.2.2	Require reverse-domain namespacing for custom assertions	Inherited from Spec 2.2
9	`validation:no_unrecognized_custom_actions`	Spec 2.4 §18.15.4	Require reverse-domain namespacing for custom actions	Inherited from Spec 2.2
10	`validation:no_unrecognized_custom_action_parameters`	Spec 2.4 §18.15.4	Require reverse-domain namespacing for custom action parameters	Inherited from Spec 2.2 (Upgraded to v2.4 parameters)
11	`validation:discouraged_legacy_action_parameters`	Spec 2.4 §18.15.4	Warn on legacy parameter placement	Inherited from Spec 2.2 (Warn)
12	`validation:review_ratings_datasource`	Spec 2.4 §18.28.3	Prevent `reviewRatings` on human entry	Inherited from Spec 2.2
13	`validation:ingredient_relationship_values`	Spec 2.4 §18.16.3	Require valid ingredient relationship values	Inherited from Spec 2.2
14	`validation:ingredient_v3_no_active_manifest`	Spec 2.4 §18.16.12.4.3	Flat ingredient `validationResults` rule	Inherited from Spec 2.2
15	`validation:ingredient_v3_mandatory_validation_results`	Spec 2.4 §18.16.12.4.3	Linked ingredient `validationResults` rule	Inherited from Spec 2.2
16	`validation:no_deprecated_claim_fields`	Spec 2.4 §10.2.3.2, Spec 2.4 §10.2.1	Prohibit top-level `specVersion` in claims	Added in Spec 2.4 (Replaces Spec 2.2 claim style)

#	Rule ID	Spec Section	Description	Source / Upgrades & Replacements
17	`validation:ingredient_v3_choice`	Spec 2.4 §18.16.12.3	Enforce mutual exclusion between `activeManifest` and `digitalSourceType`	Added in Spec 2.4 (Restricts Spec 2.2 ingredient format)
18	`validation:alternative_content_representation_choice`	Spec 2.4 §15.10.3.2.7	Require valid choice of parameters for alternative content representations	Added in Spec 2.4
19	`validation:forbidden_labels_in_external_references`	Spec 2.4 §15.10.3.2.2, Spec 2.4 §18.24.1	Prevent external reference assertions from pointing to forbidden internal labels	Added in Spec 2.4
20	`validation:no_url_in_hashes`	Spec 2.4 §18.5, Spec 2.4 §18.6	Prohibit deprecated `url` fields in hash assertions (`c2pa.hash.data`, `c2pa.hash.bmff.v3`)	Added in Spec 2.4 (Replaces Spec 2.2 hash format)
21	`validation:all_actions_included_opened`	Spec 2.4 §18.15.3	Require `allActionsIncluded: true` when a claim generator opens and resaves an asset	Added in Spec 2.4
22	`validation:opened_action_ingredient_reference`	Spec 2.4 §15.10.3.2.3, Spec 2.4 §18.15.4.7	Require every `c2pa.opened` action to contain exactly one ingredient reference	Added in Spec 2.4
23	`validation:no_box_hash_for_tiff`	Spec 2.4 §18.7.3.3, Spec 2.4 §15.10.1.2	Prohibit deprecated box-hashes on TIFF-based media formats	Added in Spec 2.4 (Replaces Spec 2.2 TIFF hash style)

B.2. Conformance Program Version Layer

B.2.1. Conformance Program 0.1 (`conformance-program-0.1.yml`)

Policy requirements established by Conformance Program 0.1 (1 Rule).

#	Rule ID	Description	Source / Status
1	`validation:mandatory_dst_for_perceptible_transformations`	Require a `digitalSourceType` on all C2PA-defined perceptible transformation actions (e.g. edits, crops)	Defined in Program 0.1

B.2.2. Conformance Program 0.2 (`conformance-program-0.1.yml` + `conformance-program-0.2.yml`)

Policy requirements for Conformance Program 0.2 (4 Rules Total: 1 inherited from Program 0.1 + 3 new rules in Program 0.2).

#	Rule ID	Description	Source / Inheritance	Gating / Scope
1	`validation:mandatory_dst_for_perceptible_transformations`	Require DST on all perceptible edit actions	Inherited from Program 0.1	Enforced for all Program 0.2 submissions
2	`validation:mandatory_all_actions_included`	Require `allActionsIncluded` field to be explicitly populated on all actions assertions	Added in Program 0.2	Enforced for all Program 0.2 submissions
3	`validation:mandatory_spec_version`	Require `claim_generator_info.specVersion` to be present and match expected version	Added in Program 0.2	Gated: Only enforced for Spec 2.4+ submissions
4	`validation:no_dst_for_opened_action`	Prohibit `digitalSourceType` on `c2pa.opened` actions	Added in Program 0.2	Gated: Only enforced for Spec 2.4+ submissions

B.3. Structural Integrity Layer (`integrity-structural.yml`)

Binary well-formedness and cryptographic signature validation rules (4 Rules).

#	Rule ID	Description	Source / Status
1	`validation:well_formed_data_present`	Check if validation results exist in the asset analysis report for structural analysis	Defined in Structural
2	`validation:well_formed_success`	Ensure no JUMBF structural or container malformation errors exist	Defined in Structural
3	`validation:valid_data_present`	Check if validation results exist in the asset report for cryptographic integrity analysis	Defined in Structural
4	`validation:valid_success`	Validate signature certificates, trust anchors, and JUMBF box hash matches	Defined in Structural

B.4. Summary Layer & Inheritance Matrix

Layer Name	Variant Target	Base Rules	Inherited Rules	Total Rules in Layer
Specification Layer	**Spec Version 2.2**	15 Rules	—	**15 Rules**
Specification Layer	**Spec Version 2.4**	8 Rules	15 Rules (from Spec 2.2)	**23 Rules**
Conformance Program Layer	**Program 0.1**	1 Rule	—	**1 Rule**
Conformance Program Layer	**Program 0.2**	3 Rules	1 Rule (from Program 0.1)	**4 Rules**
Structural Integrity Layer	**All Submissions**	4 Rules	—	**4 Rules**